

Migraciones

Cómo versionar el schema de tu base de datos
como si fuera código

01

El repositorio base

Por qué arrancamos desde un repo nuevo este módulo

¿Por qué un repo nuevo?

El problema

- Sus blogs personales divergieron
- Cada quien tiene su propio schema y tablas distintas
- Cosas diferentes en cada repo
- Si practicamos sobre eso, la clase se convierte en debugging individual

La solución

- Un repo plantilla en común para todos
- Mismo stack, mismo schema, mismos datos de prueba
- Las prácticas del módulo funcionan igual para todos
- Es el mismo tipo de sistema que van a usar en su proyecto final

Sistema de Solicitudes Internas

Una app de gestión donde empleados hacen solicitudes y administradores las gestionan. No es el tema del módulo — es el contexto para que las prácticas tengan sentido.

React

Cliente · puerto 5173

Express

Servidor · puerto 3000

PostgreSQL

Base de datos · puerto
5432

Docker

Solo para la DB



Solo Docker corre la base de datos — el servidor y el cliente corren directamente en tu máquina con Node.js.

Fork y puesta en marcha



Primero: haz un Fork en GitHub, no clones el mío directamente. Ese fork es tu entrega.

1

Hacer fork en GitHub

GitHub → Fork

2

Clonar tu fork

```
git clone <url-de-tu-fork>
```

3

Levantar la base de datos

```
docker compose up -d
```

4

Copiar variables de entorno

```
cp server/.env.example server/.env  
cp client/.env.example client/.env
```

5

Instalar dependencias

```
cd server && npm install  
cd ../client && npm install
```

6

Ejecutar migraciones

```
cd server && npm run migrate
```

7

Cargar datos de prueba

```
npm run seed
```

8

Levantar servidor y cliente

```
cd server && npm run dev  
cd client && npm run dev
```

Usuarios de prueba

Abre <http://localhost:5173> y entra con cualquiera de estas cuentas:

Email	Contraseña	Rol
admin@example.com	Password123.	admin
alice@example.com	Password123.	user
bob@example.com	Password123.	user
charlie@example.com	Password123.	user

02

Migraciones

Qué son, por qué existen y cómo funciona el mecanismo

¿Qué pasa sin migraciones?

Escenario: trabajan en equipo y alguien necesita agregar una columna status a requests.



El mensaje de WhatsApp

"Oye, corre esto en tu base de datos"

Alguien no lo ve. Alguien lo corre dos veces. Nadie sabe quién ya lo corrió.



El DROP TABLE

La solución nuclear: borrar todo y recrear desde cero.

Funciona... hasta que hay datos reales en producción.



Sin historial

¿El schema de producción tiene la columna? ¿Cuándo se agregó?
¿Quién lo hizo?

No hay forma de saberlo.

El schema es código — ponlo en el repo

Antes

Schema de la DB vive fuera del repositorio — en la cabeza de cada quien, en un mensaje de Slack, o en ningún lado.



Después

Cada cambio al schema es un archivo en el repo: versionado, ordenado, con autoría y fecha.

```
server/src/db/migrations/  
|-- 1746000001000_create-users.js  
|-- 1746000002000_create-areas.js  
|-- 1746000003000_create-user-areas.js  
|-- 1746000004000_create-sessions.js  
|-- 1746000005000_create-requests.js  
|-- 1746000006000_create-request-history.js
```

Cómo funciona npm run migrate

1

Archivo numerado

Cada archivo es un cambio atómico al schema. El timestamp garantiza el orden de ejecución.

```
1746000005000_create-requests.js

exports.up = (pgm) => {
  pgm.createTable('requests', {
    id: 'id',
    title: { type: 'text',
      nullable: true },
    ...
  })
}
exports.down = (pgm) => {
  pgm.dropTable('requests')
}
```

2

Tabla pgmigrations

La base de datos recuerda qué migraciones ya corrieron. Así sabe cuáles faltan.

```
SELECT name, run_on
FROM pgmigrations
ORDER BY run_on;

-- name
-- 1746000001000_create-users
-- 1746000002000_create-areas
-- 1746000003000_create-user-areas
-- ...
```

3

npm run migrate

Aplica solo las migraciones que no están en pgmigrations, en orden. Es seguro correrlo varias veces.

```
$ npm run migrate

> Migrating:
1746000005000_create-requests
> Migrated:
1746000005000_create-requests

# Si no hay nada nuevo:
> No migrations to run!
```

Las reglas del equipo

Acordar estas reglas es lo que hace que el repo sea la fuente de verdad para todos.

Nunca modificas el schema a mano

Sin ALTER TABLE en pgAdmin, sin cambios directos en la DB. Si está en la DB pero no en una migración, no existe para el equipo.

Nunca editas una migración ya aplicada

Una vez que alguien corrió esa migración, editarla crea inconsistencias. Si te equivocaste, creas una nueva que corrige.

Un cambio = un archivo de migración

Agrega columna → nueva migración. Crea tabla → nueva migración. Cambia tipo de dato → nueva migración.

El repo siempre puede arrancar desde cero

npm run migrate debe reproducir el schema completo en cualquier máquina. Ese es el contrato.

Lo que van a hacer hoy

Parte 1

Leer las migraciones existentes

- ¿Qué tabla crea 1746000005000_create-requests.js?
- ¿Para qué sirve exports.down?
- ¿Por qué el timestamp es tan largo?

Parte 2

Crear la migración: priority

- Agregar columna priority a requests
- VARCHAR(20) · NOT NULL · default 'normal'
- Aplicar, revertir y volver a aplicar

Parte 3

Mostrar priority en el frontend

- La columna ya existe — conectarla a la UI
- Tú decides dónde: lista, detalle, formulario
- Lo importante: el dato viene del backend

Parte 4 — Investigación libre

Agregar phone a users

- Sin instrucciones — investiga la sintaxis tú
- Agrega phone a la tabla users
- Muéstrala en UserList



Entrega del repositorio

Para que pueda revisar tu trabajo necesitas hacer una de estas dos cosas:

1

Visibilidad pública

Settings → Danger Zone
→ Change visibility
→ Make public

— 0 —

2

Agregar colaborador

Settings → Collaborators
→ Add people
→ username:

rafaelh-yn



Sin acceso no puedo calificar — verifica antes de la fecha de entrega.